

California Legal RAG Project Report

Date: April 30, 2026

1. Project Overview

This project is a custom hybrid legal question-answering system built over:

- California Constitution
- California Education Code Title 1

The goal was to build a more grounded and less hallucinatory legal QA system by combining:

- vector retrieval,
- legal-aware metadata filtering,
- knowledge graph expansion,
- citation-grounded answer generation.

This was implemented as a step-by-step pipeline rather than using a full orchestration framework like LangChain or LlamaIndex.

2. What We Built

We built the following components:

1. Corpus ingestion pipeline from local legal PDFs 2. Corpus normalization into structured JSONL 3. Legal-aware chunking with article/section anchors 4. Chroma vector index for semantic retrieval 5. Retrieval CLI for debugging and inspection 6. Citation-grounded answer generation 7. Neo4j knowledge graph extraction and loading pipeline 8. Topic hierarchy generation using an HNMF-style approach 9. KG-assisted retrieval mode for answer generation

- 1 Validation runner for batch evaluation
- 2 Question sets and report artifacts for testing

3. Dataset Scope

The current dataset scope is:

- California Constitution documents: 32
- California Education Code Title 1 documents: 480
- Total source documents parsed: 512

4. Data and Chunking Statistics

Based on `data/chunks/chunk\_stats.json` and the generated artifacts:

- Input documents: 512
- Output chunks: 3,948
- Constitution chunks: 353

- Education Code chunks: 3,595
- Chunk target size: 1,800 characters
- Chunk overlap: 250 characters

Generated data files:

- ``data/corpus/docs.jsonl``
- ``data/chunks/chunks.jsonl``
- ``data/chunks/chunk_stats.json``
- ``data/chroma/chroma.sqlite3`` plus Chroma index binaries

5. Tools and Technologies Used

Core language and environment

- Python
- Local virtual environment
- Git and GitHub

Retrieval and embeddings

- ChromaDB
- ``sentence-transformers``
- Embedding model: ``all-MiniLM-L6-v2``

LLM and generation

- OpenAI API
- Models used in the project notes and scripts:

- ``gpt-5`` - ``gpt-5-mini`` in some recovery or earlier test flows

Knowledge graph

- Neo4j AuraDB Free
- Cypher

Document processing and NLP utilities

- ``pypdf``
- ``scikit-learn``

Topic modeling

- TF-IDF + NMF approximation for an HNMF-style topic hierarchy
- Optional TELF/HNMFk adapter path in ``step7b_telf_hnmfk_topics.py``

6. Pipeline Scripts

The main scripts currently in `src/` are:

- `step1\_build\_corpus.py`
- `step2\_legal\_chunks.py`
- `step3\_build\_index.py`
- `step4\_query\_rag.py`
- `step5\_rag\_answer.py`
- `step6\_build\_neo4j\_kg.py`
- `step7\_hnmf\_topics.py`
- `step7b\_telf\_hnmfk\_topics.py`
- `run\_validation\_batch.py`
- `ui\_rag\_app.py`

7. Architecture Summary

This project follows a two-lane hybrid architecture.

Vector lane

1. Ingest legal source files 2. Normalize into structured documents 3. Chunk into legal retrieval units 4. Embed chunks 5. Store embeddings and metadata in Chroma 6. Retrieve top-k context 7. Generate grounded answer with citations

Graph lane

1. Start from chunked legal text 2. Extract entities, legal relations, and structural references 3. Store them in Neo4j 4. Expand retrieval context using graph relations 5. Pass expanded evidence into final answer generation

Result:

- hybrid RAG = semantic retrieval + graph-based relational expansion

8. Knowledge Graph Design

The KG pipeline creates nodes such as:

- `Document`
- `Chunk`
- `Entity`
- `Section`
- `Article`
- `Topic`

Key relationship types include:

- `HAS\_CHUNK`
- `MENTIONS`

- `RELATION`
- `MENTIONS\_SECTION`
- `IN\_ARTICLE`
- `IN\_TOPIC`
- `SUBTOPIC\_OF`

According to the current project report, the Neo4j graph reached:

- more than 11,000 nodes
- more than 23,000 relationships

9. Validation and Evaluation Work

We created and ran batch validation over a 50-question evaluation set.

Completed validation run:

- Run name: `validation\_crossfix`
- Timestamp: `20260413\_135527`
- Questions: 50
- Modes evaluated:

- vector - kg

- `top\_k`: 6
- `kg\_expand\_k`: 8
- Model: `gpt-5`
- Rows produced:

- 50 vector answers - 50 KG answers

- Runtime errors:

- vector: 0 - kg: 0

Evaluation artifacts include:

- `data/eval/validation\_questions.csv`
- `data/eval/results/validation\_questions\_20260413\_125840\_results.csv`
- `data/eval/results/validation\_questions\_20260413\_125840\_results.jsonl`
- `data/eval/results/validation\_questions\_20260413\_125840\_summary.json`
- `data/eval/results/validation\_crossfix\_20260413\_135527\_results.csv`
- `data/eval/results/validation\_crossfix\_20260413\_135527\_results.jsonl`
- `data/eval/results/validation\_crossfix\_20260413\_135527\_summary.json`

We also created an additional second question set:

- `data/eval/validation\_questions\_set2.csv`
- `data/eval/validation\_questions\_set2.docx`
- `data/eval/validation\_questions\_set2.pdf`

10. What the System Demonstrated

The project has already demonstrated:

1. End-to-end legal QA pipeline working 2. Grounded answer generation with citations 3. Support for semantic retrieval with metadata filters 4. KG-assisted retrieval mode 5. Parallel KG extraction with checkpoint/resume 6. A reusable validation workflow with multiple evaluation artifacts

A useful qualitative result is that the system stayed grounded in California legal materials better than a generic chat baseline in several examples, especially where generic chat drifted into U.S. constitutional answers.

11. Key Design Decisions

Important design choices in this project:

1. JSONL as the intermediate artifact format - simple - deterministic - easy to debug
2. Chunk-level legal metadata - improves filtering - improves traceability - supports article/section-aware retrieval
3. Chroma as the vector baseline - fast semantic retrieval - simple local setup - good for top-k experimentation
4. Neo4j for graph expansion - supports legal structure and relation traversal - improves explainability - helps retrieve structurally related context
5. Citation-grounded generation - reduces unsupported claims - improves legal traceability

12. Challenges Encountered

Main challenges noted in the project:

1. Neo4j Aura connectivity issues 2. Long runtime and cost for KG extraction 3. Temporary OpenAI safety-triggered failures on some chunks 4. KG expansion occasionally introducing noisy context

Mitigations used:

- retries and backoff
- checkpoint and resume
- batch processing
- lower concurrency when needed
- optional use of smaller model variants in recovery flows

13. Current Status

Current status of the project:

- Corpus build: complete

- Chunking: complete
- Chroma index: complete
- Retrieval CLI: complete
- Citation-grounded QA: complete
- Topic hierarchy: complete
- KG-assisted answer mode: implemented
- KG ingestion: implemented and run in batches
- Validation runs: completed
- Additional test-question assets: created

14. Important Project Documents and Artifacts

Core project documents:

- ``README.md``
- ``INTERVIEW_PROJECT_REPORT.md``
- ``questions_to_check_on_llm.pdf``

Evaluation and comparison artifacts:

- ``data/eval/chatgpt_answers_from_pdf.json``
- ``data/eval/chatgpt_vector_kg_comparison.csv``
- ``data/eval/chatgpt_vector_kg_comparison.json``
- ``data/eval/validation_questions.csv``
- ``data/eval/validation_questions_set2.csv``
- ``data/eval/validation_questions_set2.docx``
- ``data/eval/validation_questions_set2.pdf``

Data artifacts:

- ``data/corpus/docs.jsonl``
- ``data/corpus/read_errors.jsonl``
- ``data/chunks/chunks.jsonl``
- ``data/chunks/chunk_stats.json``
- ``data/chroma/chroma.sqlite3``

15. Summary for Report or Interview

This project is a custom California legal RAG system built over the California Constitution and Education Code Title 1. It processes 512 source documents into 3,948 legal-aware chunks, indexes them in Chroma for semantic retrieval, and augments retrieval with a Neo4j knowledge graph. The system supports citation-grounded answer generation, graph-assisted context expansion, topic hierarchy generation, and

batch validation. It has already completed working end-to-end validation runs with 50 questions across both vector-only and KG-assisted modes, with zero runtime errors in the latest recorded run. The project is functionally complete as a hybrid legal QA pipeline and is now strong enough to present as a reportable capstone or interview project.